Since you're starting on a **new Ubuntu 24.04 system**, it's best to install everything from scratch using the **official ESP-IDF installation method**. This is the same setup you'll use for your **IIIT Bangalore Vaaman board (ESP32)**.

Step 1: Update Ubuntu

Open Terminal and run:

```
sudo apt update
sudo apt upgrade -y
```

Step 2: Install Required Packages

Install all dependencies:

```
sudo apt install -y git wget flex bison gperf python3 python3-pip python3-venv \
cmake ninja-build ccache libffi-dev libssl-dev dfu-util libusb-1.0-0
```

Verify versions:

```
git --version
python3 --version
cmake --version
ninja --version
```

Step 3: Create ESP Directory

```
mkdir -p ~/esp
cd ~/esp
```

Step 4: Clone ESP-IDF

Clone the official ESP-IDF repository:

```
git clone --recursive https://github.com/espressif/esp-idf.git
```

This creates:

```
~/esp/
    esp-idf/
```

Step 5: Install ESP-IDF Tools

Go into the ESP-IDF directory:

```
cd ~/esp/esp-idf
```

Run:

```
./install.sh esp32
```

This downloads and installs:

- Xtensa GCC toolchain
- Python packages
- OpenOCD
- CMake support
- ESP flashing tools
- esptool.py

It may take **10–20 minutes**, depending on your internet connection.

Step 6: Load the ESP-IDF Environment

Each new terminal session needs the environment loaded:

```
source ~/esp/esp-idf/export.sh
```

You should see a message indicating the environment has been set up successfully.

Step 7: Make It Automatic (Recommended)

To avoid running `source` every time, add it to your shell configuration:

```
echo 'source ~/esp/esp-idf/export.sh' >> ~/.bashrc
```

Reload it:

```
source ~/.bashrc
```

Now every new terminal will have `idf.py` available automatically.

Step 8: Verify Installation

Run:

```
idf.py --version
```

You should see something like:

```
ESP-IDF v5.x.x
```

Also check:

```
python --version
```

```
esptool.py version
```

Step 9: Connect the Vaaman Board

Plug the board into USB.

Check the detected serial device:

```
ls /dev/ttyUSB*
```

or

```
ls /dev/ttyACM*
```

If nothing appears, check:

```
dmesg | tail -30
```

Step 10: Give Serial Port Permission

Add your user to the `dialout` group:

```
sudo usermod -aG dialout $USER
```

Then **log out and log back in**, or reboot.

Verify:

```
groups
```

You should see:

```
dialout
```

Step 11: Test with the Blink Example

Go to the example project:

```
cd ~/esp/esp-idf/examples/get-started/blink
```

Set the target:

```
idf.py set-target esp32
```

Build:

```
idf.py build
```

Flash (replace the port if needed):

```
idf.py -p /dev/ttyACM0 flash
```

Open the serial monitor:

```
idf.py -p /dev/ttyACM0 monitor
```

Exit the monitor with:

```
Ctrl + ]
```

Recommended Directory Structure

After installation, your home directory should look like this:

```
Home
│
├── esp
│   └── esp-idf
```

```
│       ├── components
│       ├── examples
│       ├── export.sh
│       ├── install.sh
│       └── tools
│
└── Projects
    ├── Blink
    ├── UART
    ├── GPIO
    └── Your_Project
```

Daily Workflow

Once everything is installed:

1. Connect the Vaaman board.
2. Open a terminal (if you didn't add `export.sh` to `.bashrc`, run `source ~/esp/esp-idf/export.sh`).
3. Go to your project:

```
cd ~/Projects/Your_Project
```

4. Build:

```
idf.py build
```

5. Flash:

```
idf.py -p /dev/ttyACM0 flash
```

6. Monitor:

```
idf.py -p /dev/ttyACM0 monitor
```

One improvement over the official method

Instead of cloning the entire ESP-IDF repository with all history, you can do a **shallow clone**, which is faster and uses less disk space:

```
git clone --recursive --depth=1 https://github.com/espressif/esp-idf.git
```

This is a good choice for a fresh development machine unless you specifically need the repository history.